

Breaking the Compression Barrier: Cross-Architecture Compression Boundary Learning via Reverse Regrowth

Zhaocen Liu

University of British Columbia
Vancouver, British Columbia, CA
liu0411@student.ubc.ca

Satvik Praveen

University of South Florida
Tampa, Florida, USA
satvikpraveen@usf.edu

Yi Sheng

University of South Florida
Tampa, Florida, USA
sheng1@usf.edu

Abstract

Model compression is critical for deploying networks on resource-constrained edge devices. While pruning-based methods can significantly reduce model size, they often suffer from abrupt performance collapse beyond a sparsity threshold, making it difficult to identify the feasible compression limit of the model. To address this challenge, we propose a boundary-Learning reverse regrowth framework, BRIDGE, that reformulates compression as a constructive boundary-search problem. Unlike forward pruning, our method first drives the model to an extremely sparse state to expose the collapse region, and then selectively regenerates the critical structure to restore performance. The proposed framework employs a hierarchical regeneration strategy, including coarse-grained layer selection and fine-grained regeneration parameter selection, to accurately identify which parameters require recovery. Experiments show that our method can recover models from the brink of collapse on both CNNs and Transformer architectures, demonstrating its architecture independence. BRIDGE achieves a performance improvement of up to 1.49% in unstructured pruning and up to 4.77% in structured pruning. These results demonstrate that reverse regeneration can effectively extend the compression limit while maintaining stable performance. The source code is available at <https://github.com/EnumaCaliber/BRIDGE>.

1 Introduction

Modern architectures, including convolutional and Transformer-based models, have achieved remarkable success across a wide range of intelligent applications [He et al. 2016; Vaswani et al. 2017]. However, deploying these models on resource-constrained edge platforms, such as mobile processors [Sandler et al. 2019], embedded systems [Lin et al. 2020], and medical devices [Shoaran et al. 2018], remains challenging because their computational, memory, and energy demands often exceed the available hardware resources. As a result, network pruning has become one of the most widely adopted model compression techniques, substantially reducing model size and computational cost while preserving accuracy close to that of the original dense model [Cheng et al. 2023; Lee et al. 2026]. Despite this success, existing pruning methods are primarily designed to optimize model accuracy under

a predefined sparsity budget, leaving a fundamental question unanswered: *what is the maximum feasible compression a model can sustain before its performance collapses?*

Existing pruning methods, whether structured [He et al. 2017; Li et al. 2017; Liu et al. 2017] or unstructured [Guo et al. 2016; Han et al. 2015], operate within a common forward sparsification paradigm “large-to-small”: a dense model is progressively compressed while its accuracy is monitored. Although different approaches employ diverse pruning criteria, such as magnitude [Han et al. 2015], gradient [Lee et al. 2021], or second-order information [LeCun et al. 1990], they differ mainly in what is removed, not in how the compression boundary is identified. As sparsity approaches the compression boundary, model performance frequently exhibits an abrupt and highly nonlinear degradation, which we refer to as the **performance cliff** [Ai et al. 2025; Ma et al. 2026]. Once this cliff is crossed, the forward pruning trajectory provides little information about where the true compression boundary lies, forcing practitioners to rely on repeated trial-and-error to estimate the maximum feasible sparsity [Frantar and Alistarh 2023]. Consequently, existing pruning methods improve compression decisions within the same forward search paradigm, yet leave the boundary identification problem fundamentally unresolved [Hoeffler et al. 2021]. We argue that identifying the compression boundary should be formulated as a boundary-learning problem, rather than a conventional pruning problem, motivating a fundamentally different search paradigm for model compression.

These observations motivate a different perspective on architecture compression. **First**, we observe that performance cliffs are strategy-independent: both one-shot and iterative pruning exhibit abrupt accuracy collapse across representative convolutional and Transformer architectures. This suggests that the compression boundary is an intrinsic property of the model rather than an artifact of a particular pruning strategy. **Second**, the collapse is highly localized. Instead of uniform degradation throughout the network, the performance cliff is primarily caused by severe representational damage in a small subset of layers, as revealed by layer-wise Structural Similarity Index (SSIM) analysis between dense and pruned feature representations.

Together, these observations suggest that identifying the compression boundary should be viewed as a boundary-learning

problem, rather than a conventional forward pruning problem. Motivated by this insight, we propose **BRIDGE**, a reverse boundary-learning framework for architecture compression. Unlike existing pruning methods that progressively sparsify a dense model while attempting to stop before the performance cliff, BRIDGE intentionally traverses the compression boundary to explicitly expose its location, and subsequently performs "reverse regrowth" to selectively restore only the minimal subset of pruned parameters or channels required to recover the minimum feasible model. By reversing the conventional large-to-small compression paradigm, BRIDGE directly identifies the compression boundary instead of approximating it through repeated forward pruning.

The regrowth process is formulated as a Markov Decision Process in which an RL-based controller allocates a limited parameter budget through a two-level hierarchical strategy. At the coarse level, SSIM-based filtering identifies the most structurally degraded layers as regrowth candidates, restricting the search space to where recovery is most needed. At the fine level, gradient-based saliency scores rank pruned weights within the selected layers, ensuring that only the most critical connections are restored. This hierarchical decomposition reduces the combinatorial action space by orders of magnitude, making policy optimization tractable under tight computational budgets. Extensive experiments across four architectures and two pruning paradigms demonstrate that BRIDGE consistently recovers accuracy in the high-sparsity regime, with search costs under 10 GPU-hours per model.

The main contributions of this work are summarized as follows:

- We identify compression boundary estimation as a previously overlooked challenge in network compression and show that conventional forward pruning is fundamentally ill-suited for boundary identification due to its monotonic paradigm and performance-cliff behavior.
- We propose BRIDGE, a reverse boundary-learning framework that reformulates network compression as a constructive boundary-learning problem by intentionally traversing the compression boundary and selectively recovering the minimum feasible model through reverse regrowth.
- We demonstrate that BRIDGE consistently identifies compression boundaries across representative convolutional and Transformer architectures, providing a unified framework for architecture-agnostic boundary-Learning compression while achieving competitive compression performance.

2 Related Works And Motivation

Observation 1: Performance cliffs persist across pruning strategies, pruning paradigms, and networks. As

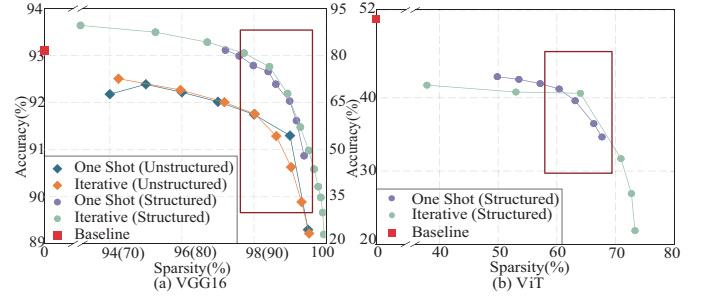


Figure 1: Performance cliffs consistently emerge across different pruning strategies and architectures: (a) VGG16; (b) ViT.

shown in Figure 1, we evaluate both one-shot and iterative pruning on representative convolutional and Transformer architectures. For VGG16 [Simonyan and Zisserman 2015], both structured and unstructured pruning are considered, whereas for ViT [Dosovitskiy et al. 2021] we focus on structured pruning, which is the dominant choice for practical Transformer acceleration. A consistent observation across all settings is the emergence of a distinct **performance cliff**. Model accuracy remains relatively stable over a wide sparsity range, but deteriorates abruptly once a critical sparsity threshold is reached [Jaiswal et al. 2023]. For VGG16, the cliff appears at approximately 98% sparsity under unstructured pruning and around 90% under structured pruning. In contrast, ViT exhibits a much earlier collapse at approximately 60% sparsity, with an even narrower transition region. This behavior is likely related to the stronger inter-layer coupling and lower structural redundancy of Transformer architectures, making them more sensitive to aggressive structured pruning. More importantly, despite differences in pruning schedule, sparsity type, and architecture, the performance cliff consistently emerges. Although the exact compression boundary varies across models and pruning settings, the recurring cliff-like behavior suggests that performance collapse is not an artifact of a particular pruning strategy, but rather a common phenomenon under extreme model compression.

Motivation 1: Revisiting Forward Sparsification. Network pruning has been extensively studied as an effective approach for neural network compression. Existing methods can be broadly categorized into iterative pruning [Han et al. 2015], importance-based pruning [Guo et al. 2016; Lee et al. 2019], and dynamic sparse training [Evci et al. 2020]. More recently, pruning techniques have been extended from convolutional networks to Transformer architectures through structured pruning of attention heads and feed-forward channels [Chen et al. 2021; Yu et al. 2022], as well as dynamic token and module pruning [Rao et al. 2021]. Although these

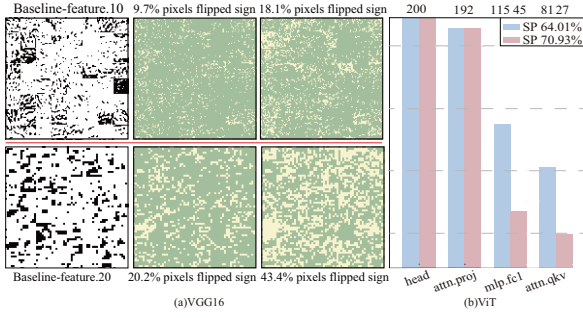


Figure 2: Under extreme sparsity: (a) sign pattern disruption in intermediate features of VGG16. Green indicates preserved sign and yellow indicates sign flip relative to the baseline. While most activations remain stable before collapse, a significant increase in sign flips occurs after collapse. (b) The ratio of surviving channels in ViT is categorized by channel type. Only two channel types show significant changes across the collapse point, while the rest remain stable.

methods differ substantially in pruning criteria, optimization objectives, and training dynamics, each of them follows the same fundamental paradigm: starting from a dense model, parameters are progressively removed while attempting to preserve model accuracy. This forward sparsification paradigm has proven highly effective under moderate sparsity, where network redundancy remains abundant and local pruning decisions are generally sufficient to maintain performance. However, under extreme sparsity, both prior studies [Frankle et al. 2021; Hoefler et al. 2021; Tanaka et al. 2020; Zhang et al. 2025] and our Observation 1 consistently reveal the emergence of a sharp *performance cliff*, where accuracy deteriorates abruptly after a critical sparsity threshold. Notably, this phenomenon is observed across different pruning schedules, sparsity types, and representative convolutional and Transformer architectures, suggesting that it is not tied to a particular pruning strategy. Consequently, the primary challenge in the extreme-compression regime shifts from identifying *which* parameters should be removed to determining *where* the compression boundary lies. Existing pruning methods improve parameter selection within the same forward sparsification paradigm, yet none is explicitly designed to identify or exploit the compression boundary, leaving this fundamental problem largely unexplored.

Observation 2: Performance Collapse Is Highly Localized. Unlike the gradual degradation observed under moderate sparsity, performance collapse is not uniformly distributed across the network but is concentrated in a small subset of layers. To investigate this phenomenon, Fig. 2(a) visualizes the sign consistency of feature maps in two representative VGG16 layers by comparing the dense baseline with a pre-collapse model (94.24%) and a post-collapse model (99.53%). Before crossing the performance cliff, the feature

maps remain largely consistent with the dense baseline, with only a small fraction of activations changing sign. After collapse, however, sign flips increase dramatically, indicating severe representational disruption in these layers. This observation is consistent with the corresponding low SSIM values, suggesting that only a few layers experience substantial structural degradation, while the majority of the network remains relatively stable. A similar phenomenon is observed in ViT. As shown in Fig. 2(b), when sparsity increases from the pre-collapse model (64.01%) to the post-collapse model (70.93%), the additional structural changes are concentrated in only two module types, whereas the remaining modules exhibit little variation. Despite their architectural differences, both CNNs and Transformers therefore exhibit highly localized changes after crossing the performance cliff, indicating that performance collapse is dominated by a small number of structurally critical components rather than uniform degradation across the entire network.

Motivation 2: Bridging Pruning and Representation Analysis. Recovering a model after crossing the compression boundary requires answering two complementary questions: *what* should be modified, and *where* recovery should be performed. Existing research has addressed these two questions separately. Pruning methods estimate parameter importance using magnitude, gradient, saliency, or second-order information to determine what weights or channels should be removed [Han et al. 2015; Hassibi and Stork 1992; LeCun et al. 1990; Lee et al. 2019; Molchanov et al. 2017], while layer-adaptive methods further allocate sparsity according to layer sensitivity [Lee et al. 2021]. In parallel, representation similarity measures, such as SVCCA, CKA, and SSIM, have been widely used to quantify feature consistency for model analysis, knowledge distillation, and network compression [Hinton et al. 2015; Kornblith et al. 2019; Raghu et al. 2017; Romero et al. 2015; Wang et al. 2004]. However, these two research directions have largely evolved independently. Existing pruning methods optimize parameter selection without considering representational degradation, whereas representation similarity is primarily used for post-hoc analysis rather than guiding model optimization [Klabunde et al. 2025; Li et al. 2024]. A unified framework that integrates pruning optimization with representation-guided recovery remains unexplored [Garg and Torra 2026; Schmitt et al. 2024; Wu et al. 2025]. Our second observation establishes this connection by revealing that performance collapse is highly localized, enabling representation similarity to directly guide targeted reverse regrowth.

Motivation 3: From Sparse Maintenance to Boundary Recovery. Regrowth has long been recognized as an effective mechanism for improving sparse neural networks. Early work introduced connection splicing to restore mistakenly pruned weights, while subsequent dynamic sparse

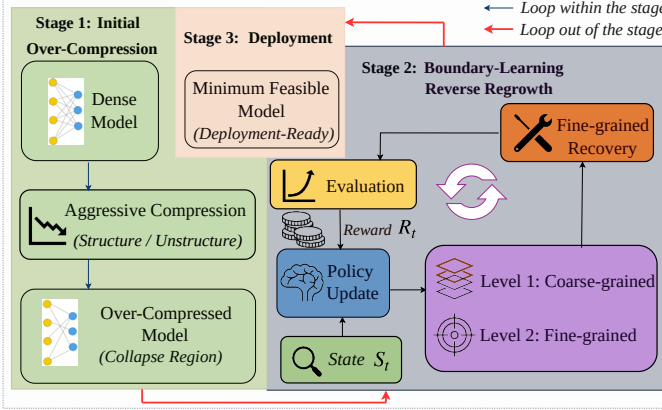


Figure 3: BEIDGE: Boundary-Learning Reverse Regrowth Framework. The framework first over-compresses the model to expose the compression boundary, then performs a hierarchical reverse regrowth process to restore critical structures, and identifies the minimum feasible model for deployment.

training methods, such as SET [Mocanu et al. 2018] and RigL [Evci et al. 2020], periodically regrow connections to maintain network connectivity and improve optimization quality [Wang et al. 2020]. Although these methods differ in implementation, they share a common objective: using regrowth to maintain a functional sparse network throughout training. Our problem is fundamentally distinct: rather than maintaining sparse-network functionality within a safe operating regime, we target models that have already crossed the performance cliff. Existing regrowth methods are not designed to identify the compression boundary or perform targeted recovery after such collapse. We therefore use regrowth as a recovery mechanism that restores only the critical connections required to return the model to a feasible operating region.

3 Framework

The overall pipeline of BRIDGE consists of three stages, as illustrated in Fig. 3. We detail each stage in the following subsections.

3.1 Stage I: Initial Over-Compression

We progressively prune the dense model to increasing sparsity levels via iterative and one-shot pruning, obtaining the pruning baseline and identifying the collapse region. Pruning criteria can be categorized by the order of information they exploit: zero-order (magnitude), first-order (gradient), and second-order (curvature), and BRIDGE is order-independent: regrowth requires only an importance measure of higher order than that used for pruning.

Collapse Region Identification: To automatically identify the onset of collapse, we compute the discrete second-order difference of accuracy with respect to sparsity. Let $A(s_i)$ denote the accuracy at the i -th sparsity level. We define the collapse onset as the smallest sparsity s^* satisfying: $\Delta^2 A(s^*) = A(s^{*+1}) - 2A(s^*) + A(s^{*-1}) < -\epsilon$, where ϵ is a predefined threshold (set to 0.5 in our experiments). This criterion captures the transition from approximately linear degradation to sharp nonlinear collapse. The sparsity level s^* serves as the entry point for the subsequent reverse regrowth process.

3.2 Stage II: Boundary-Aware Reverse Regrowth

Starting from the sparsity level proximal to the collapse region identified in Stage I, the framework enters a closed iterative regrowth loop, which we formulate as a Markov Decision Process (MDP) [Benmeziane et al. 2021; Zoph and Le 2017]. At each step, an RL-based controller observes the current model state S_t and outputs a hierarchical regrowth action that determines where and how to restore model capacity. The regrown model is then fine-tuned and evaluated, producing a reward R_t that drives policy updates. The controller is trained to maximize the expected reward over all layer-wise decisions: $J(\theta_c) = \mathbb{E}_{P(a_{1:T}, \theta_c)}[R_T]$, where θ_c denotes the controller parameters and $a_{1:T}$ is the sequence of layer-wise allocation actions. Since R_T is non-differentiable with respect to θ_c , we optimize $J(\theta_c)$ via the REINFORCE policy-gradient algorithm [Zoph and Le 2017]. This loop repeats until convergence, progressively restoring critical capacity while preserving high sparsity. Each component of the loop is detailed below.

Step 1 - State: At each episode step t , the state S_t is represented by a 3-dimensional context vector:

$$S_t = \left[\frac{p+1}{N+1}, \frac{\text{cap}_l}{\sum_l \text{cap}_l}, \frac{r}{N_{\text{restore}}} \right], \quad (1)$$

where p is the current layer index, cap_l is the number of pruned weights in layer l , and r is the remaining unallocated budget. The three dimensions are designed to provide the controller with the minimal yet sufficient information for making informed regrowth decisions, encoding the relative layer depth $\frac{p+1}{N+1}$, the layer’s share of pruned weights $\frac{\text{cap}_l}{\sum_l \text{cap}_l}$ (reflecting structural damage and recovery potential), and the remaining budget fraction $\frac{r}{N_{\text{restore}}}$ (preventing over-allocation to early layers).

Step 2 - Action: Hierarchical Reverse Regrowth. We decompose the action a_t into two levels: a coarse-grained stage that selects the target layer, thereby reducing the search space and improving efficiency, and a fine-grained stage that

specifies which parameters or channels to regrow within it, ensuring effectiveness.

① **Level 1 - Coarse-grained layer selection stage:** Motivated by Observation 2, we compute the SSIM [Wang et al. 2004] between the feature maps of the sparse model and the dense baseline at each layer. Layers with the lowest SSIM values are preferentially selected as candidate regrowth layers. However, SSIM relies on local spatial statistics of the feature maps and therefore becomes unreliable for layers with small or absent spatial dimensions. To cover this regime, we incorporate cosine similarity as a complementary metric. Similar to SSIM, cosine similarity measures the degree of collapse relative to a dense model, but it is scale-invariant and independent of spatial statistics, so it remains valid even when SSIM fails: values close to 1 indicate a complete representation, while negative values indicate that most activations have undergone sign inversion relative to a dense model and are close to collapse. Table 1 validates the situation at layers where SSIM fails: layers 13 and 14 exhibit negative cosine similarity and a sign-flip rate higher than 55%, while layer 15 maintains positive cosine similarity (0.5444) and a much lower sign-flip rate (28.12%). This confirms that cosine similarity can accurately track the layer’s representation to determine if it has collapsed. Therefore, these two metrics characterize structural damage from a complementary perspective and can be flexibly combined according to the architecture.

Table 1: Cosine similarity is consistent with layer damage (sign flip rate); negative values indicate collapse.

Layer	Cosine	Sign Flip (%)
layers.13	−0.1139	55.73
layers.14	−0.1405	56.38
layers.15	0.5444	28.12

② **Level 2 - Fine-grained regrowth stage:** After identifying degradation-sensitive layers, we perform fine-grained regrowth by ranking candidate units with a higher-order importance measure and restoring only the highest-ranked ones. Since pruning may remove parameters that later prove critical, richer information is needed to distinguish these mistakenly discarded units from the broader pruned set. We therefore reserve higher-order criteria for targeted regrowth. **Step 3 - Reward and policy update:** After each regrowth step, the model is finetuned and evaluated, and the reward is defined as the normalized accuracy improvement over the pruning baseline at the corresponding sparsity level, i.e., $R_t = (A_{\text{regrowth}} - A_{\text{baseline}}(s))/100$, where $A_{\text{baseline}}(s)$ is interpolated from the pre-computed accuracy-sparsity curve. The controller is updated by REINFORCE with an exponential moving-average baseline to reduce variance and entropy regularization for exploration.

3.3 Stage III: Deployment

Upon convergence of Stage II, the regrown model satisfying the target performance threshold is selected as the final deployment-ready output. Deployment is an essential stage of BRIDGE: the ultimate goal of boundary-aware compression is to produce models that actually run faster and smaller on resource-constrained hardware. Our deployment evaluation is conducted on structured-pruned models, since structured sparsity translates directly into hardware benefits. The resulting model executes directly on standard dense kernels, delivering real acceleration without any specialized sparse library, a property that is particularly valuable for Transformer architectures, where sparse-kernel support remains limited. The core value of BRIDGE at deployment time lies in extending the compression limit itself. Sparsity levels that were previously accuracy-infeasible, where forward pruning has already collapsed, become usable after BRIDGE. Consequently, under the same accuracy requirement, BRIDGE allows a model to be deployed at a higher sparsity than conventional pruning permits, which directly means fewer parameters, fewer FLOPs, faster inference, and lower latency on device. To faithfully emulate realistic resource-constrained scenarios, we validate this on a Raspberry Pi, a representative edge platform with limited compute and memory resources and no GPU acceleration.

4 Experiments

4.1 Experiments Setting

Common protocol. Unless otherwise specified, all fine-tuning uses AdamW [Loshchilov and Hutter 2017] (with a learning rate of 3×10^{-4} , weight decay 10^{-2}) with early stopping, and accuracy is evaluated on the full test set.

1) Models and Pretraining: We evaluate ResNet20 [He et al. 2016], EfficientNetB0 [Tan and Le 2019], ShuffleNetV2 [Ma et al. 2018], and VGG16 on CIFAR-10 [Krizhevsky 2009], and ViT on the larger Tiny-ImageNet, covering design paradigms from residual connections and depthwise convolutions to self-attention. All models are pretrained from scratch with SGD (momentum 0.9, weight decay 5×10^{-4} , lr 0.1 with cosine annealing, 200 epochs, batch size 128) and standard augmentation.

2) Pruning: The baseline combines iterative pruning (15 rounds, removing 30% of remaining weights per round with 40 fine-tuning epochs each; 10% per round for ViT due to its earlier and sharper collapse) with one-shot pruning at sparsity levels near the collapse edge (fine-tuned up to 400 epochs). For structured pruning, since channel indices are re-numbered after each round, we maintain a list of *alive indices* mapping each current position to its dense-model index; recording output channels suffices, as coupled input channels are implicitly determined. This mapping enables

Table 2: Accuracy Improvement of BRIDGE over Strong Pruning Baselines in Extreme Sparsity Regimes. Results on the first four column groups are on CIFAR-10; the last column group reports VGG16 on Tiny-ImageNet.

Type	ResNet20			EfficientNetB0			ShuffleNetV2			VGG16			VGG16 (Tiny-ImageNet)		
	Sp.(%)	Base.(%)	Regr.(%)	Sp.(%)	Base.(%)	Regr.(%)	Sp.(%)	Base.(%)	Regr.(%)	Sp.(%)	Base.(%)	Regr.(%)	Sp.(%)	Base.(%)	Regr.(%)
LAMP [Lee et al. 2021]	98.08	83.26	-	98.73	87.26	87.44	95.00	91.29	-	99.00	91.30	-	-	-	-
	98.26	81.21	81.34	98.93	86.84	87.23	97.43	91.04	-	99.08	91.21	-	99.02	52.58	52.89
	98.36	80.67	80.91	99.03	86.41	87.14	98.11	90.74	-	99.15	90.70	90.81	99.15	51.96	52.16
	98.46	80.14	80.35	99.12	85.55	86.85	98.49	90.22	90.41	99.25	90.30	90.43	99.25	51.55	51.65
	98.56	79.59	80.12	99.23	84.61	86.05	98.57	90.12	90.21	99.35	89.90	90.34	99.35	50.73	50.99
	98.74	78.33	79.11	99.33	83.71	85.20	98.67	90.02	90.16	99.37	89.82	90.44	99.42	50.35	50.62
	98.83	77.63	78.31	99.43	82.61	83.59	98.76	89.92	90.22	99.43	89.58	90.17	99.44	50.22	50.54
Search Cost (GPU-h)	3.47			9.57			5.26			6.64			9.82		

the regrowth stage to reconstruct the local computation graph and restore each channel together with its structurally coupled channels.

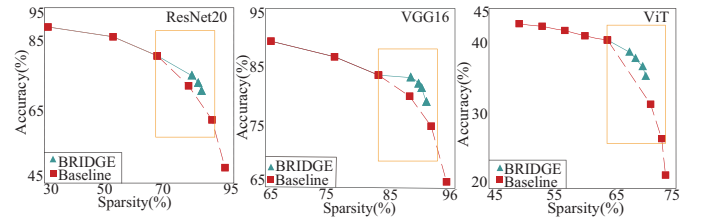
3) BRIDGE Framework: The controller (hidden size 64, Adam, lr 3×10^{-4}) selects layer-wise allocation from $K = 11$ proportions in $[0, 1]$ and, in the iterative setting, a budget fraction from $K_b = 5$ options in $[0.1\%, 0.5\%]$ of prunable weights. The entropy coefficient decays from 0.40 to 0.04 over the first 40% of episodes ($\tau = 0.005$). Training runs up to 300 episodes per iteration with early stopping on reward stagnation. Post-regrowth finetuning runs 50 (one-shot) or 40 (iterative) epochs with regrown weights zero-initialized; models surpassing the baseline are saved as candidates, and one-shot candidates undergo a final full fine-tuning (cosine annealing with warmup, up to 400 epochs). If none surpasses the baseline, the highest-reward model is retained.

4) Hardware: Edge deployment targets structured-pruned models only (Section 3.3) on a Raspberry Pi 5 (8 GB RAM), with FP32 ONNX export and ONNX Runtime 1.26 (CPU, 4 threads). We profile dense, pruned, and regrown checkpoints of VGG16, ResNet20, and ViT, measuring per-image latency, parameters, FLOPs, and file size.

4.2 Exploration of the BRIDGE

Consistent gains under extreme sparsity. Fig. 4 shows that under structured pruning, BRIDGE achieves pronounced recovery near the collapse boundary: the improvement on ResNet20 reaches +4.77%, and on VGG16 the BRIDGE-recovered curve even trends *upward* in the high-sparsity range while the baseline declines sharply, indicating that the recovered channels are precisely those critical to model performance. Tab 2 further confirms this under unstructured pruning, where BRIDGE consistently improves accuracy over strong pruning baselines with gains growing as sparsity increases, reaching up to +1.49% on EfficientNetB0. On the larger Tiny-ImageNet dataset, VGG16 still shows improvement, albeit smaller; we attribute this to the higher task complexity, which leaves less recoverable redundancy after aggressive pruning. These results indicate that the advantage of BRIDGE lies not in

benign compression regimes, but precisely where conventional pruning begins to collapse.

**Figure 4: Structured pruning versus BRIDGE**

Recovery near the collapse boundary. More importantly, these improvements concentrate around the collapse boundary, where performance transitions from gradual degradation to abrupt failure and pruning baselines fall below critical usability thresholds. Under structured pruning (Fig. 4), VGG16 falls below 80% accuracy, and BRIDGE lifts it back above 80%, close to the low-sparsity baseline. Tab 2 shows the same behavior under unstructured pruning: ShuffleNetV2 and VGG16 both drop below the 90% threshold at extreme sparsity, whereas BRIDGE restores both above it. This is precisely one of the core objectives of BRIDGE: it identifies the collapse boundary and regrows the model back into the usable regime, converting configurations that forward pruning renders unusable into deployment-feasible ones.

Architecture-independent recovery. A key advantage of BRIDGE is architecture independence, which Fig. 4 and Tab 2 jointly support from complementary perspectives. Fig. 4 shows that BRIDGE extends beyond convolutional networks: on ViT, whose collapse arrives markedly earlier and more sharply (Observation 1) and whose prunable units are attention/MLP dimensions rather than convolutional channels, BRIDGE still accurately localizes the collapse region and performs targeted recovery. Tab 2 further shows that four CNNs with fundamentally different design paradigms, despite exhibiting distinct degradation behaviors, all benefit from BRIDGE. All in all, four CNNs and one Transformer with two datasets,

and two pruning paradigms, BRIDGE maintaining stable effectiveness across all cases, reinforcing our central claim that model compression should be treated as a boundary-aware learning problem rather than a purely destructive pruning process.

Framework efficiency. Benefiting from the two-stage design, BRIDGE reduces the search space by orders of magnitude (e.g., from 10^{82} to 10^4 on EfficientNetB0), evaluating fewer than 2700 configurations in total. As shown in Tab 2, its search cost stays below 10 GPU-hours on a single GPU for all models, comparable to ENAS [Pham et al. 2018] (7.7 GPU-hours on CIFAR-10), despite the latter requiring no per-episode fine-tuning on a structurally simpler search problem.

Table 3: Deployment results on Raspberry Pi 5.

Model	Config	Sp.(%)	Acc.(%)	Lat.(ms)	Speedup	FLOPs(M)
ResNet20	L1-norm	78.46	72.76	0.402	3.2×	3.01
	BRIDGE	82.49	73.49	0.397	3.2×	2.25
VGG16	L1-norm	88.40	80.28	0.418	42.2×	7.94
	BRIDGE	90.36	81.62	0.408	43.2×	8.37
ViT	L1-norm	64.96	37.98	1.909	3.4×	53.65
	BRIDGE	67.55	38.92	1.741	3.7×	42.75

Edge device deployment: Tab 3 shows that structured pruning delivers real acceleration on device, reaching up to 43.2 times speedup on VGG16. More importantly, BRIDGE does not trade speed for recovery: across all three architectures, it simultaneously achieves higher sparsity, higher accuracy, and lower latency than the forward-pruning baseline, indicating that regrowth reallocates capacity more effectively than forward pruning itself. Notably, although the regrown VGG16 incurs marginally more FLOPs, it still runs faster with far fewer parameters, confirming that theoretical FLOPs are a poor proxy for on-device latency and that the regrown structures are more hardware-friendly [Yang et al. 2018]. These results show BRIDGE enabling deployment-ready models under extreme structured sparsity while extending the collapse boundary.

4.3 Ablation Studies

Effectiveness of RL-based component: We compare BRIDGE with greedy and fixed-proportion baselines using random and saliency-based allocation. As shown in Table 4, BRIDGE consistently performs best, indicating that saliency alone is insufficient. Its advantage arises from combining over-compression to expose the boundary with RL-based search to identify the parameters requiring restoration. Thus, effective recovery depends on the joint use of boundary-aware constraints and adaptive allocation.

Generalization of pruning and regrowth order: While our main experiments use zero-order pruning with first-order

Table 4: Comparison between BRIDGE, fixed proportion(P), and pure greedy(G)(S: Saliency, R: Random)

Model	Method	Sparsity (%)			
		99.43	99.33	99.23	99.13
VGG16	P+R	89.66 ± 0.16	89.80 ± 0.20	90.03 ± 0.11	90.03 ± 0.03
	P+S	89.74 ± 0.19	90.00 ± 0.15	90.24 ± 0.11	90.30 ± 0.08
	G+R	89.47 ± 0.21	89.61 ± 0.23	89.73 ± 0.13	89.73 ± 0.17
	G+S	89.51 ± 0.17	89.78 ± 0.11	89.78 ± 0.11	89.96 ± 0.27
	BRIDGE	90.09 ± 0.16	90.37 ± 0.02	90.46 ± 0.12	90.52 ± 0.21
Efficient-NetB0	P+R	81.93 ± 0.42	82.93 ± 0.58	83.69 ± 0.77	84.52 ± 0.59
	P+S	83.13 ± 0.67	84.78 ± 0.53	85.55 ± 0.53	86.14 ± 0.58
	G+R	81.17 ± 0.18	81.65 ± 0.15	82.14 ± 0.13	82.26 ± 0.16
	G+S	81.11 ± 0.27	81.97 ± 0.30	82.53 ± 0.85	83.89 ± 1.09
	BRIDGE	83.35 ± 0.34	85.05 ± 0.22	85.84 ± 0.29	86.41 ± 0.62

Table 5: Accuracy gains under different pruning-regrowth order combinations.

Prune order	Model	Regrowth order			
		1st order		2nd order	
		Sp. (%)	Gain (%)	Sp. (%)	Gain (%)
	0th	90.08	+3.39	89.96	+3.27
	VGG16	91.36	+1.89	90.65	+2.90
		81.38	+4.77	83.25	+3.50
	ResNet20	83.93	+3.85	84.01	+3.29
	1st			90.15	+5.70
				91.15	+2.84
	ResNet20			84.69	+6.05
				85.46	+5.05

regrowth, the framework generalizes to any combination where the regrowth order exceeds the pruning order. We validate this on VGG16 and ResNet20: as Tab. 5 shows, both first and second-order regrowth outperform the corresponding pruning baselines. Moreover, on ViT, applying second-order regrowth to a first-order pruned model yields a +4.48% improvement at 69.48% sparsity, confirming that the order-based regrowth criterion is architecture-independent.

Different Layer Selection: We adopt SSIM as the primary metric for layer selection and cosine similarity as a supplementary one, evaluating both configurations on ResNet20 and EfficientNetB0. As shown in Tab. 6, similarity-guided selection consistently outperforms the pruning baseline across both architectures. Adding cosine similarity improves ResNet20 by up to 0.71% with modest additional cost, but provides no consistent benefit on EfficientNetB0 while substantially increasing search time. These results indicate that SSIM offers the best overall accuracy-cost trade-off, whereas cosine similarity is beneficial only for selected architectures.

5 Conclusion

This paper addresses the drastic performance degradation of conventional pruning methods under extreme sparsity. We propose BRIDGE, a boundary-Learning reverse regrowth framework that reformulates compression as a constructive boundary-search problem: the model is first over-compressed to expose its compression boundary, and critical structures are then selectively recovered through hierarchical regrowth.

Table 6: Accuracy and search cost comparison by different layer selection combinations.

Model	Sp.(%)	Baseline (%)	Acc.(%)		Cost (GPU-h)	
			SSIM	SSIM+COS	SSIM	SSIM+COS
Efficie- ntNetB0	99.03	86.41	87.14	86.99	4.13	38.33
	99.13	85.41	86.78	86.74	3.23	21.24
	99.23	84.61	86.05	86.17	3.68	27.61
	99.33	83.71	85.20	85.32	8.10	31.88
	99.43	82.61	83.59	83.28	6.38	14.95
ResNet20	98.74	78.33	79.17	79.42	2.29	4.63
	98.83	77.63	78.31	79.02	2.60	5.39
	98.93	76.85	77.50	77.55	2.52	3.19

Experiments across diverse architectures show that BRIDGE restores near-collapsed models to usable performance without additional inference overhead, achieving gains of up to 4.77%. More importantly, BRIDGE is architecture-agnostic, delivering consistent improvements on both CNN and Transformer architectures. BRIDGE offers a new perspective on neural network compression: boundary-learning recovery rather than purely destructive pruning, breaking the compression barrier.

References

- Mengting Ai, Tianxin Wei, Sirui Chen, and Jingrui He. 2025. NIRVANA: Structured pruning reimaged for large language models compression. *arXiv preprint arXiv:2509.14230* (2025).
- Hadjer Benmezziane, Kaoutar El Maghraoui, Hamza Ouarnoughi, Smail Niar, Martin Wistuba, and Naigang Wang. 2021. A Comprehensive Survey on Hardware-Aware Neural Architecture Search. *arXiv preprint arXiv:2101.09336* (2021).
- Tianlong Chen, Yu Cheng, Zhe Gan, Lu Yuan, Lei Zhang, and Zhangyang Wang. 2021. Chasing Sparsity in Vision Transformers: An End-to-End Exploration. *arXiv:2106.04533* [cs.CV]
- Hongrong Cheng, Miao Zhang, and Javen Qinfeng Shi. 2023. A survey on deep neural network pruning-taxonomy, comparison. *Analysis, and Recommendations* 2 (2023).
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *arXiv:2010.11929* [cs.CV] <https://arxiv.org/abs/2010.11929>
- Utku Evci, Trevor Gale, Jacob Menick, Pablo Samuel Castro, and Erich Elsen. 2020. Rigging the Lottery: Making All Tickets Winners. In *International Conference on Machine Learning*. 2943–2952.
- Jonathan Frankle, Gintare Karolina Dziugaite, Daniel Roy, and Michael Carlin. 2021. Pruning Neural Networks at Initialization: Why Are We Missing the Mark?. In *ICLR*.
- Elias Frantar and Dan Alistarh. 2023. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International conference on machine learning*. PMLR, 10323–10337.
- Sonakshi Garg and Vicens Torra. 2026. PrunePrivyTune: Accelerating Language Models with Pruning and Differentially Private Fine-Tuning. *Machine Learning* 115, 5 (2026), 108.
- Yiwen Guo, Anbang Yao, and Yurong Chen. 2016. Dynamic Network Surgery for Efficient DNNs. *arXiv:1608.04493* [cs.NE] <https://arxiv.org/abs/1608.04493>
- Song Han, Jeff Pool, John Tran, and William J Dally. 2015. Learning Both Weights and Connections for Efficient Neural Networks. In *NeurIPS*.
- Babak Hassibi and David G. Stork. 1992. Second Order Derivatives for Network Pruning: Optimal Brain Surgeon. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 5. 164–171.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Yihui He, Xiangyu Zhang, and Jian Sun. 2017. Channel Pruning for Accelerating Very Deep Neural Networks. In *ICCV*.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531* (2015).
- Torsten Hoefler et al. 2021. Sparsity in Deep Learning: Pruning and growth for efficient inference and training. *JMLR* (2021).
- Ajay Jaiswal, Shiwei Liu, Tianlong Chen, Zhangyang Wang, et al. 2023. The emergence of essential sparsity in large pre-trained models: The weights that matter. *Advances in Neural Information Processing Systems* 36 (2023), 38887–38901.
- Max Klabunde, Tobias Schumacher, Markus Strohmaier, and Florian Lemmerich. 2025. Similarity of neural network models: A survey of functional and representational measures. *Comput. Surveys* 57, 9 (2025), 1–52.
- Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. 2019. Similarity of Neural Network Representations Revisited. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, Vol. 97. PMLR, 3519–3529.
- Alex Krizhevsky. 2009. *Learning Multiple Layers of Features from Tiny Images*. Technical Report. University of Toronto.
- Yann LeCun, John S Denker, and Sara A Solla. 1990. Optimal Brain Damage. In *NeurIPS*.
- Gilhyeon Lee, Seonggeun Kim, Dongjun Lee, Kyungmin Goh, and Hyun Kim. 2026. Towards efficient language giants: A comprehensive survey on structural optimizations and compression techniques for large language models. *Neural Networks* (2026), 108900.
- Jaeho Lee, Sejun Park, Sangwoo Mo, Sungsoo Ahn, and Jinwoo Shin. 2021. Layer-Adaptive Sparsity for the Magnitude-based Pruning Criterion. In *International Conference on Theoretical Representations (ICLR)*.
- Namhoo Lee, Thalaiyasingam Ajanthan, and Philip H S Torr. 2019. SNIP: Single-Shot Network Pruning Based on Connection Sensitivity. In *ICLR*.
- Hao Li, Asim Kadav, Igor Durdanovic, Hanan Samet, and Hans Peter Graf. 2017. Pruning Filters for Efficient ConvNets. In *ICLR*.
- Yuqi Li, Yao Lu, Junhao Dong, Zeyu Dong, Chuanguang Yang, Xin Yin, Yihao Chen, Jianping Gou, Yingli Tian, and Tingwen Huang. 2024. Sglp: A similarity guided fast layer partition pruning for compressing large deep models. *arXiv preprint arXiv:2410.14720* (2024).
- Ji Lin, Wei-Ming Chen, Yujun Lin, John Cohn, Chuang Gan, and Song Han. 2020. MCUNet: Tiny Deep Learning on IoT Devices. *arXiv:2007.10319* [cs.CV] <https://arxiv.org/abs/2007.10319>
- Zhuang Liu, Jianguo Li, Zhiqiang Shen, Gao Huang, Shuicheng Yan, and Changshui Zhang. 2017. Learning Efficient Convolutional Networks through Network Slimming. In *ICCV*.
- Ilya Loshchilov and Frank Hutter. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101* (2017).
- Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. 2018. ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design. In *Proceedings of the European Conference on Computer Vision (ECCV)*.
- Ziyang Ma, Zuchao Li, Lefei Zhang, Gui-Song Xia, Bo Du, Liangpei Zhang, and Dacheng Tao. 2026. Phase transitions in large language model compression. *npj Artificial Intelligence* 2, 1 (2026), 21.
- Decebal Constantin Mocanu, Elena Mocanu, Peter Stone, Phuong H. Nguyen, Madeleine Gibescu, and Antonio Liotta. 2018. Scalable Training of Artificial Neural Networks with Adaptive Sparse Connectivity Inspired by Network Science. In *Nature Communications*, Vol. 9. 2383.
- Dmitry Molchanov, Arsenii Ashukha, and Dmitry Vetrov. 2017. Variational Dropout Sparsifies Deep Neural Networks. In *ICML*.
- Hieu Pham, Melody Y. Guan, Barret Zoph, Quoc V. Le, and Jeff Dean. 2018. Efficient Neural Architecture Search via Parameter Sharing. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*. PMLR, 4095–4104.
- Maithra Raghu, Justin Gilmer, Jason Yosinski, and Jascha Sohl-Dickstein. 2017. SVCCA: Singular Vector Canonical Correlation Analysis for Deep Learning Dynamics and Interpretability. In *Advances in Neural Information Processing Systems (NeurIPS)*. 6076–6085.
- Yongming Rao, Wenliang Zhao, Benlin Liu, Jiwen Lu, Jie Zhou, and Cho-Jui Hsieh. 2021. DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification. *arXiv:2106.02034* [cs.CV] <https://arxiv.org/abs/2106.02034>
- Adriana Romero, Nicolas Ballas, Samira Ebrahimi Kahou, Antoine Chassang, Carlo Gatta, and Yoshua Bengio. 2015. FitNets: Hints for Thin Deep Nets. In *International Conference on Learning Representations (ICLR)*.
- Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2019. MobileNetV2: Inverted Residuals and Linear Bottlenecks. *arXiv:1801.04381* [cs.CV] <https://arxiv.org/abs/1801.04381>
- Jonas Schmitt, Ruiping Liu, Junwei Zheng, Jiaming Zhang, and Rainer Stiefelhagen. 2024. Comb, Prune, Distill: Towards Unified Pruning for Vision Model Compression. In *2024 IEEE 27th International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2413–2419.
- Mahsa Shoran, Benyamin Allahgholizadeh Haghi, Milad Taghavi, Masoud Farivar, and Azita Emami-Neyestanek. 2018. Energy-Efficient Classification for Resource-Constrained Biomedical Applications. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 8, 4 (2018), 693–707. doi:10.1109/JETCAS.2018.2844733
- Karen Simonyan and Andrew Zisserman. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Mingxing Tan and Quoc V. Le. 2019. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In *Proceedings of the International Conference on Machine Learning (ICML)*.

- Hidekazu Tanaka, Daniel Kunin, Daniel L. Yamins, and Surya Ganguli. 2020. Pruning Neural Networks without Any Data by Iteratively Conserving Synaptic Flow. In *Advances in Neural Information Processing Systems*, Vol. 33. 6377–6389.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. In *NeurIPS*.
- Chaoqi Wang, Guodong Zhang, and Roger Grosse. 2020. Picking Winning Tickets Before Training by Preserving Gradient Flow. In *ICLR*.
- Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. 2004. Image Quality Assessment: From Error Visibility to Structural Similarity. *IEEE Transactions on Image Processing* 13, 4 (2004), 600–612. doi:10.1109/TIP.2003.819861
- Zimeng Wu, Jiaxin Chen, and Yunhong Wang. 2025. Unified knowledge maintenance pruning and progressive recovery with weight recalling for large vision-language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 8550–8558.
- Tien-Ju Yang, Andrew Howard, Bo Chen, Xiao Zhang, Alec Go, Mark Sandler, Vivienne Sze, and Hartwig Adam. 2018. Netadapt: Platform-aware neural network adaptation for mobile applications. In *Proceedings of the European conference on computer vision (ECCV)*. 285–300.
- Fang Yu, Kun Huang, Meng Wang, Yuan Cheng, Wei Chu, and Li Cui. 2022. Width & Depth Pruning for Vision Transformers. In *AAAI Conference on Artificial Intelligence (AAAI)*, Vol. 2022.
- Qiaozhe Zhang, Ruijie Zhang, Jun Sun, and Yingzhuang Liu. 2025. How Sparse Can We Prune A Deep Network: A Fundamental Limit Perspective. arXiv:2306.05857 [stat.ML] <https://arxiv.org/abs/2306.05857>
- Barret Zoph and Quoc V. Le. 2017. Neural Architecture Search with Reinforcement Learning. arXiv:1611.01578 [cs.LG] <https://arxiv.org/abs/1611.01578>